

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI -590 014



**DEPARTMENT OF
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**

MINI PROJECT (BAI586)

REPORT ON

“Resume Parsing with Google Gemini LLM”

Submitted By

RAGHUNANDHAN R

SURYAPRAKASHGOWDA H S

KAIFIYA S

(4EG22AI032)

(4EG22AI046)

(4EG23AI405)

**Under the Guidance of
Asst. Prof. KHAIRUNNESA**

Dept. of AIML



**GOVERNMENT ENGINEERING COLLEGE CHALLAKERE
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
Ballary road, Challakere, Chitradurga- 577522**

(Affiliated to Visvesvaraya Technological University, Belagavi, Recognized by AICTE,
New Delhi and Approved by Government of Karnataka)

2024-2025

GOVERNMENT ENGINEERING COLLEGE

CHALLAKERE- 577522



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ENGINEERING

CERTIFICATE

This is to Certify that the project, titled “**Resume Parsing with Google Gemini LLM**” carried out by **RAGHUNANDHAN R (4EG22AI032) SURYAPRAKASHGOWDA H S (4EG22AI046) KAIFIYA S (4EG23AI405)** a bonafide student of **Department of AIML Engineering, GEC, Challakere** in partial fulfilment for the award of **Bachelor of Engineering in Artificial Intelligence and Machine Learning** of **GEC, Challakere** during the year **2024-25**. It is certified that all correction/suggestions indicated for internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements for the said degree.

GUIDE

Asst. Prof. KHAIRUNNESA

HOD

Asst. Prof. SAHANA V

Name of the Examiner

Signature

Examiner 1:

Examiner 2:

GOVERNMENT ENGINEERING COLLEGE

CHALLAKERE- 577522



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ENGINEERING

DECLARATION

We **RAGHUNANDHAN R (4EG22AI032)** **SURYAPRAKASHGOWDA H S (4EG22AI046)** **KAIFIYA S (4EG23AI405)** the student of 5th semester **Bachelor of Engineering in Artificial Intelligence and Machine Learning, GEC, Challakere**, hereby declare that the project report titled **“Resume Parsing with Google Gemini LLM”** has been prepared and presented by me and submitted in partial fulfilment of the course requirements for the award of degree in **Bachelor of Engineering in Artificial Intelligence and Machine Learning, GEC, Challakere**, during the year **2024- 2025**. Further the content of the report has not been submitted previously for the award of any degree or diploma to any other university.

Place: Challakere

Date: 27/12/2024

RAGHUNANDHAN R

4EG22AI032

SURYAPRAKASHGOWDA H S

4EG22AI046

KAIFIYA S

4EG23AI405

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the progress and completion of any task would be incomplete without the mention of the people, who made it possible, whose constant guidance and encouragement ground my efforts with success.

We consider it as a privilege to express my sincere gratitude and respect to all those who guided and inspired me.

We express my sincere thanks and gratitude to my guide **KHAIRUNNESA**, Assistant professor, Department of AIML Engineering, GEC, Challakere, for her constant guidance and suggestions. Her incessant encouragement and invaluable support have been of immense help.

It's a great privilege to express my respect to **SAHANA V**, Assistant Professor and Head, Department of AIML Engineering, GEC, Challakere, who had been great source of inspiration towards taking up this project and its successful completion.

We also thankful to **Teaching** and **Non-teaching** staff for giving me the support and encouragement that was necessary for the completion of this Project.

Cordially

RAGHUNANDHAN R **4EG22AI032**

SURYAPRAKASHGOWDA H S **4EG22AI046**

KAIFIYA S **4EG23AI405**

CONTENTS	PAGE NO.
List of Figures	
List of Tables	
1. ABSTRACT	
CHAPTER 1	01
INTRODUCTION	
AIMS & OBJECTIVES	02
CHAPTER 2	
LITERATURE SURVEY	03-07
CHAPTER 3	
SOFTWARE AND HARDWARE REQUIREMENT	08-24
CHAPTER 4	
RESULTS	25-27
FUTURE WORK	28-29
CHAPTER 5	
CONCLUSION AND FUTURE SCOPE	30
BIBLIOGRAPHY	31

LIST OF FIGURES

Figure No.	Description	Page No.
3.1	BLOCK DIAGRAM	10
3.2	FLOW CHART	12
4.1	USER INTERFACE WITH RESULTS	27

LIST OF TABLES

Table No.	Description	Page No.
2.1	Summary of Literature Review	7

ABSTRACT

The **Resume Parsing with Google Gemini LLM** project is a web application built to simplify and enhance the process of extracting key information from resumes. Using a Flask backend and powered by Google Gemini LLM, a generative AI model, the application processes resume in PDF format to extract structured data. This data includes personal information, skills, educational qualifications, work experience, certifications, and language proficiencies. By automating what is traditionally a manual and time-consuming task, the application serves as an efficient tool for both recruiters and job seekers.

Upon uploading a resume, the application reads the document using PDF processing libraries and analyzes the content with the AI model. The processed data is then returned as a structured JSON file, which is displayed on a user-friendly interface. This interface organizes the information for easy review, helping recruiters quickly assess candidates' qualifications or enabling candidates to review their data for accuracy.

To provide a seamless user experience, the application incorporates robust error handling mechanisms. It identifies and addresses common issues, such as invalid file formats or corrupted PDFs, by providing meaningful error messages and guiding users to resolve these problems. This focus on reliability makes the tool accessible and practical for various use cases, regardless of the technical expertise of its users.

The use of generative AI through Google Gemini LLM enables the application to not only extract information but also enhance it with insights and summaries. This feature offers value beyond basic parsing, allowing for a deeper understanding of a candidate's profile and potential fit for a role. Additionally, the application's ability to process multiple resumes concurrently highlights its scalability, making it suitable for organizations of varying sizes.

By automating resume parsing and offering intelligent analysis, this project addresses a critical need in modern recruitment. Its combination of advanced AI, user-centric design, and robust functionality positions it as a powerful tool for transforming how resumes are reviewed and utilized in hiring processes.

CHAPTER 1

INTRODUCTION

In the rapidly evolving job market, the recruitment process faces increasing challenges due to the high volume of applications and the need for timely, effective candidate evaluations. As organizations strive to identify suitable candidates quickly, traditional manual resume review methods become less viable. This project addresses these challenges by developing a web-based application that utilizes Google's Gemini Large Language Model (LLM) to automate the parsing and analysis of resumes, significantly enhancing the efficiency of the hiring process.

The primary objective of this application is to transform unstructured resume data into structured, actionable insights. By extracting critical information such as contact details, skills, education, and work experience, the application streamlines the evaluation process for recruiters. Leveraging advanced natural language processing capabilities, the system allows for quick access to essential candidate information, enabling hiring managers to focus on the most qualified applicants and make informed decisions.

The application also includes a user-friendly interface that simplifies the resume upload process, catering to users with varying levels of technical expertise. Robust error handling ensures a smooth user experience, addressing potential issues such as invalid file formats or text extraction failures. By integrating Google Gemini LLM, the application not only improves the accuracy of data extraction but also offers tailored insights and recommendations, helping candidates better align their qualifications with job market demands.

AIMS & OBJECTIVES

AIMS

Speed Up Hiring: Automatically extract important information from resumes to make the recruitment process faster.

Ensure Accurate Data: Gets candidate details accurately, reducing mistakes that can happen with manual data entry.

Encourage Fair Hiring: Address biases and support diversity by effectively managing different resume formats, promoting a fairer hiring process.

OBJECTIVES

Make Hiring Faster: Automatically extract and organize candidate information from resumes to speed up the recruitment process.

Improve Accuracy and Fairness: Ensure the data is correct, reduce bias, and promote diversity in hiring, creating a fairer experience for everyone involved.

CHAPTER 2

LITERATURE SURVEY

2.1 AN AUTOMATED RESUME SCREENING SYSTEM USING NATURAL LANGUAGE PROCESSING AND SIMILARITY

The automated resume screening system leverages Natural Language Processing (NLP) and similarity measures to streamline the recruitment process. By extracting key information from resumes, such as qualifications and skills, the system reduces manual data entry, allowing recruiters to quickly gather and organize candidate details. It uses similarity scoring, like cosine similarity, to assess how closely each resume matches job descriptions, enabling efficient candidate ranking. This automation saves significant time for HR teams and enhances accuracy by minimizing human bias in evaluations. Ultimately, the faster screening process leads to improved candidate experiences, as applicants receive timely feedback, making them feel valued and engaged with the organization.

2.2 A BRIEF SURVEY OF TEXT MINING: CLASSIFICATION, CLUSTERING AND EXTRACTION TECHNIQUES

A brief survey of text mining explores three key techniques: classification, clustering, and extraction. Classification involves categorizing text into predefined labels using machine learning algorithms trained on labelled datasets, such as identifying spam emails. Clustering groups similar documents without predefined labels, uncovering natural patterns in the data, which is useful for exploratory analysis. Extraction focuses on retrieving specific information, such as named entities or key phrases, from the text. Together, these techniques enable effective analysis of large volumes of unstructured text, transforming it into actionable insights across various applications.

2.3 VECTOR SPACE MODEL. INFORMATION RETRIEVAL

The Vector Space Model (VSM) is a mathematical model used in information retrieval to represent documents and queries as vectors in a multi-dimensional space. In this model, each dimension corresponds to a unique term from the document corpus, allowing for the quantification of the relevance of documents to a given query. The similarity between a document and a query is typically measured using cosine similarity, which calculates the cosine of the angle between their respective vectors. This approach facilitates efficient retrieval of relevant documents based on keyword matching and term frequency. By representing both documents and queries as vectors, the VSM enhances the ability to rank and retrieve information effectively in large datasets.

2.4 NLP BASED EXTRACTION OF RELEVANT RESUME USING MACHINE LEARNING

This technique stated parsing of the resumes with least limit and the parser works the utilization of two or three rules which train the call and address. Scout bundles use the CV parser system for the determination of resumes. As resumes are in amazing arrangements and it has different sorts of real factors like set up and unstructured estimations, meta experiences, etc. The proposed CV parser approach gives the component extraction method from the moved CV's.

2.5 E-RECRUITMENT SYSTEM THROUGH RESUME PARSING, PSYCHOMETRIC TEST AND SOCIAL MEDIA ANALYSIS

It follows an approach of 4 stages, the first stage was to get the data (resume) and convert them into structured format and then perform the analysis using deep learning techniques. Second step includes the psychometric test where the text mining is used to generate scores for each candidate. In the third step they perform web scraping on various social media sites to get the additional information about the candidates and recommend suitable jobs to them. In the fourth step, the system will recommend the skills and requirements in which the students are lacking and help them to get recruited in the desired company.

2.6 COMBINATION OF NEURAL NETWORK AND CONDITIONAL RANDOM FIELDS FOR EFFICIENT RESUME PARSING

The techniques used in this category are neural networks and CRF to segment and extract various information from resumes. CNN model is used for segmentation and compared with a Bi LSTM model. A CRF based model is chosen for information extraction and compared with a Bi-LSTM-CNN model. They segmented and extracted several pieces of information from personal, educational, and occupational blocks. The results are promising and the output JSON file contains 23 data fields.

2.7 RESUME EVALUATION SYSTEM BASED ON AI

The "Resume Evaluation System Based on AI" employs artificial intelligence to automate the assessment of resumes. It analyzes key components like skills and qualifications to provide a comprehensive evaluation of candidates. By utilizing machine learning algorithms, the system enhances the accuracy and efficiency of recruitment, enabling recruiters to identify the best-fit candidates effectively. This AI-driven approach streamlines evaluations and supports data-driven hiring decisions.

2.8 A CV PARSER MODEL USING ENTITY EXTRACTION PROCESS AND BIG DATA TOOLS

The CV parser model utilizes advanced entity extraction techniques and big data tools to extract relevant information from resumes. It focuses on identifying key components such as personal details, skills, education, and work experience from CVs. By leveraging big data technologies, the system efficiently handles large volumes of resumes, ensuring quick and accurate parsing. The entity extraction process enhances the precision of identifying and categorizing various elements within the resumes, ultimately improving the recruitment workflow. This approach aims to streamline candidate evaluation and provide recruiters with structured, actionable insights from unstructured data.

2.9 SCRAPE KEYWORDS CONVERSION OF DIFFERENT FORMATS OF RESUMES TO TEXT

The process of scraping keywords involves extracting relevant terms and phrases from resumes to enhance candidate evaluation in recruitment. This technique focuses on identifying key skills, qualifications, and experiences that align with job requirements. Additionally, the conversion of various resume formats—such as PDF format is crucial for effective parsing and analysis. By transforming different file types into a uniform text format, recruiters can ensure consistency in data extraction and improve the accuracy of keyword identification. This combined approach streamlines the recruitment process, enabling more efficient matching of candidates to job descriptions.

2.10 AN UNSTRUCTURED TEXT ANALYTICS APPROACH FOR QUALITATIVE EVALUATION OF RESUMES

In this work, a qualitative assessment of resumes on the basis of different quality parameters using a simple text analytic based approach for a resume collection was described. The resume collection was processed for two qualitative coverage, comprehensibility, and the aspects; and extracted ratings are modified into a quality rating which is comprehensive. All the parameters were collectively uniformed into a combined 1 to 5 rating scale for associating a quality metric for resumes. The qualitative evaluation results obtained through the algorithmic approach were congruent to and were hence validated through the wisdom of crowds.

LITERATURE SURVEY				
SL NO	TITLE OF PAPER	YEAR	AUTHORS	CONTENT EXTRACTED
1.	AN AUTOMATED RESUME SCREENING SYSTEM USING NATURAL LANGUAGE PROCESSING AND SIMILARITY	2020	Chirag Daryani , et.al	- Named Entity Recognition - Automated Resume Screening
2.	A Brief Survey of Text Mining: Classification, Clustering and Extraction Techniques	2017	Mehdi Allahyari, et.al	- Clustering - Information extraction
3.	Vector Space Model. Information Retrieval	2013	Jaime Arguello	- Vector Space Model (VSM) - TF-IDF (Term Frequency-Inverse Document Frequency)
4.	NLP based Extraction of Relevant Resume using Machine Learning	2020	Nirali Bhaliya, et.al	- Parsing with lease limit - Similarity index for skill sets
5.	E-Recruitment System Through Resume Parsing, Psychometric Test And Social Media Analysis	2019	Dr.Parkavi A, et.al	- Text mining is used to generate scores - Web scraping
6.	Combination of Neural Network and Conditional Random Fields for Efficient Resume Parsing	2018	Ayishathahira C H, et.al	- Classify resume into three segments - Extract 23 different data fields

LITERATURE SURVEY				
SL NO	TITLE OF PAPER	YEAR	AUTHORS	CONTENT EXTRACTED
7.	Resume Evaluation System based on AI	2020	Rutuja Patil, et.al	- Artificial Intelligence - Classification techniques
8.	A CV Parser Model using Entity Extraction Process and Big Data Tools	2018	Papiya Das, et.al	- Convert unstructured resumes to structured - Supports multiple language
9.	Scrape keywords Conversion of different formats of resumes to text	2017	Satyaki Sanyal, et.al	- Scrape keywords - Conversion of different formats of resumes to text
10.	An Unstructured Text Analytics Approach for Qualitative Evaluation of Resumes	2015	Vinaya R. Kudtarkar, et.al	- A simple text analytic - Quality metric for resume - Comprehensive quality rating

Table. 2.1

CHAPTER 3

SOFTWARE AND HARDWARE REQUIREMENT

3.1. Software Requirements

To ensure smooth operation of the Flask Resume Parser application, here are the essential software requirements:

3.1.1. Backend Software

1. Operating System:

- Windows, Linux, or macOS (any OS that supports Python and Flask).

2. Programming Language:

- Python 3.x (Ensure you're using Python 3.7 or higher).

3. Web Framework:

- Flask: A lightweight Python web framework for routing and handling HTTP requests.
 - Install using: `pip install flask`

4. PDF Parsing Library:

- PyPDF2: For extracting text from PDF files.
 - Install using: `pip install PyPDF2`

5. Generative AI Library:

- Google Generative AI (Gemini 1.5): For parsing and analysing the resume content.
 - Install using: `pip install google-generativeai`

7. JSON:

- Standard library for parsing and generating JSON data in Python.

Frontend Libraries:

- a. HTML, CSS, JavaScript: For rendering the parsed data, displaying the pie chart, and handling basic interactions like form submission.

8. Development Tools:

- VS Code, PyCharm, or any text editor of choice for writing Python code.

3.1.2. Hardware Requirements

1. Processor:

- A multi-core processor (Intel i5 or higher recommended) for fast execution, especially when processing large resumes.

2. Memory (RAM):

- 4GB RAM minimum, 8GB or higher is recommended for better performance, especially if handling multiple resume uploads simultaneously.

3. Storage:

- 10GB of available storage for the operating system, software libraries, and resume files. Ensure there is enough space to store uploaded resumes and generated charts.

4. Network:

- Stable internet connection for accessing Google's AI API.

Block Diagram

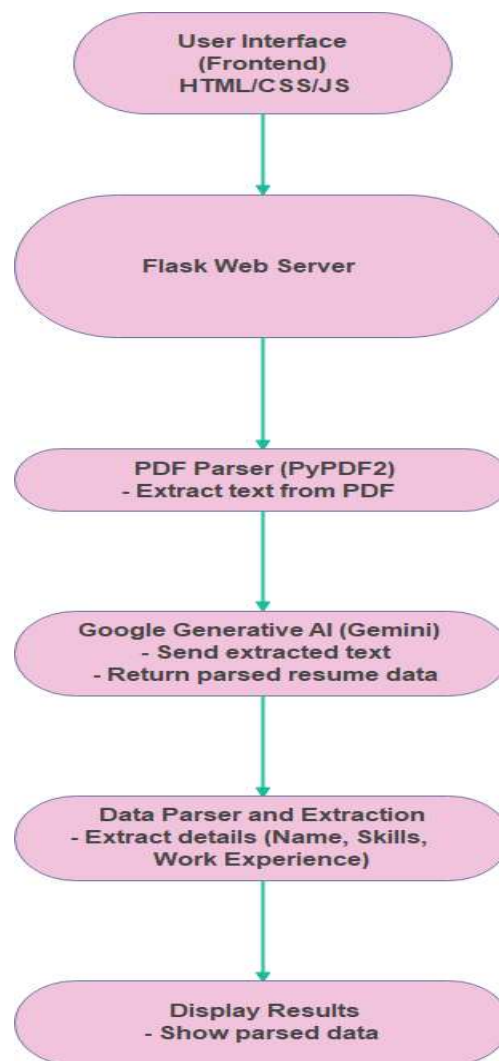


Fig.no. 3.1

User Interface (Frontend):

- The user uploads the PDF resume through an HTML form. The frontend handles user interactions like displaying the results and showing the pie chart.

Flask Web Server:

- This is the central component. It listens for HTTP requests (especially file uploads) and forwards them to the relevant backend modules. It renders the HTML response back to the user, containing parsed data and charts.

PDF Parser (PyPDF2):

- The PyPDF2 library extracts the text from the uploaded PDF resume.

Google Generative AI (Gemini):

- The text extracted from the PDF is sent to the Google Generative AI model (Gemini) for detailed parsing, which returns a structured JSON containing the parsed resume information.

Data Parser and Extraction:

- The raw data (JSON) returned by the AI model is parsed and structured to extract key details like name, contact info, skills, education, work experience, etc.

Display Results:

- The parsed data are displayed to the user via the HTML frontend.

Flowchart

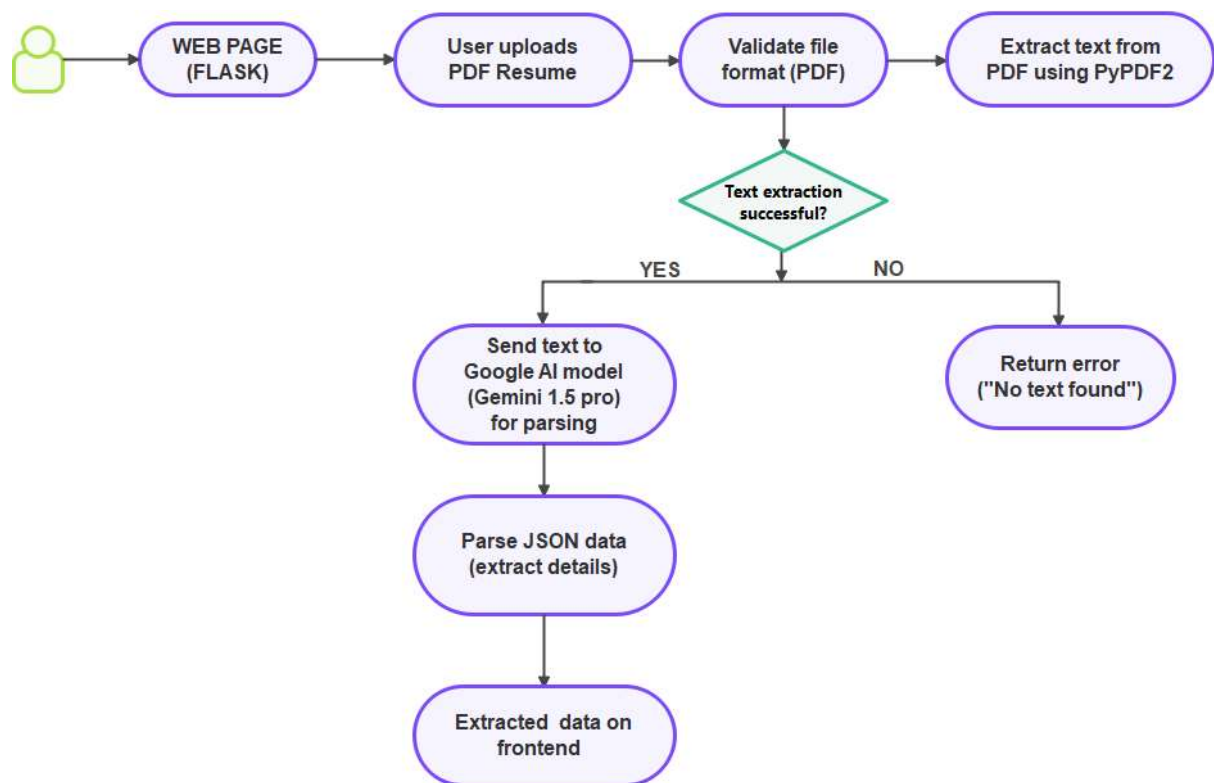


Fig.no.3.2

User Uploads PDF Resume:

- The user selects and uploads a PDF file via the web form.

Validate File Format:

- The server checks whether the uploaded file is a PDF. If it's not a PDF, an error is returned.

Extract Text from PDF:

- The text is extracted from the PDF using the PyPDF2 library.

Text Extraction Successful?

- If text extraction is successful, the process continues. If not, the system returns an error indicating no text was found in the PDF.

Send Text to Google Generative AI Model:

- The extracted text is sent to Google's **Generative AI (Gemini 1.5)** model for detailed parsing of the resume's sections (e.g., name, skills, experience).
- The AI model processes the resume and returns a structured JSON response containing the parsed details.

Parse JSON Data:

- The server parses the returned JSON and extracts specific details like name, skills, work experience, and education.

Count Resume Sections:

- The application counts the number of entries in each resume section (e.g., skills, education, work experience) to be used in generating a pie chart.

Render Parsed Data:

- The parsed data are passed to the HTML template

CODE PART

FRONTEND CODE

```
from flask import Flask, render_template, request, jsonify
from PyPDF2 import PdfReader

import google.generativeai as genai

import os

import json


# Set your API key
os.environ["GOOGLE_API_KEY"] = "AIzaSyBNz8LSlJbDP3AA8A9vxt_UPfyqbLQPZnY"
genai.configure(api_key=os.environ["GOOGLE_API_KEY"])


# app creation
app = Flask(__name__)


# load model
model = genai.GenerativeModel("models/gemini-1.5-pro")


def resumes_details(resume):

    # Prompt with detailed request
    prompt = f"""

    You are a resume parsing assistant. Given the following resume text, extract all the
    important details and return them in a well-structured JSON format.
```

The resume text:

{resume}

Extract and include the following:

- Full Name
- Contact Number
- Email Address
- Location
- Skills (Technical and Non-Technical, separately if possible)
- Education
- Work Experience (including company name, role, and responsibilities)
- Certifications
- Languages spoken
- Suggested Resume Category (based on the skills and experience)
- Recommended Job Roles (based on the candidate's skills and experience)

Return the response in JSON format.

"""

Generate response from the model

response = model.generate_content(prompt).text

return response

@app.route('/')

def index():

return render_template('index.html')

```
@app.route('/upload_resume', methods=['POST'])
def upload_resume():
    if 'resume' not in request.files:
        return jsonify({"error": "No file part"})

    file = request.files['resume']

    if file.filename == "":
        return jsonify({"error": "No selected file"})

    if file and file.filename.endswith('.pdf'):
        # Extract text from the PDF
        text = ""
        reader = PdfReader(file)
        for page in reader.pages:
            text += page.extract_text()

        # Get resume details from the model
        response = resumes_details(text)

        print(response)

        # Clean the response by removing the ```json and surrounding text
        response_clean = response.replace("```json", "").replace("```", "").strip()

        # Load the cleaned response into a dictionary
        data = json.loads(response_clean)
```

Extract details from the JSON response

full_name = data.get("Full Name", "")

contact_number = data.get("Contact Number", "")

email_address = data.get("Email Address", "")

location = data.get("Location", "")

Skills extraction

skills = data.get("Skills", {})

technical_skills = skills.get("Technical Skills", [])

non_technical_skills = skills.get("Non-Technical Skills", [])

technical_skills_str = ", ".join(technical_skills)

non_technical_skills_str = ", ".join(non_technical_skills)

Education extraction

education_list = data.get("Education", [])

education_str = "\n".join([

f"{edu.get('Degree', 'N/A')} from {edu.get('Institution', 'N/A')} (Graduated:
 {edu.get('Years', 'N/A')})*"*

for edu in education_list

])

Work Experience extraction

work_experience_list = data.get("Work Experience", [])

work_experience_str = "\n".join([

f"{job.get('Job Title', 'N/A')} at {job.get('Company Name', 'N/A')} ({job.get('Years of
Experience', 'N/A')})\nResponsibilities: {' '.join(job.get('Responsibilities', []))}*"*

for job in work_experience_list

])

```
# Certifications extraction

certifications_str = ", ".join(data.get("Certifications", []))

# Languages extraction

languages_str = ", ".join(data.get("Languages spoken", []))

# Suggested Category and Recommended Job Roles extraction

suggested_resume_category = data.get("Suggested Resume Category", "")
recommended_job_roles_str = ", ".join(data.get("Recommended Job Roles", []))

# Render the template and pass the extracted values to HTML

return render_template('index.html',
                        full_name=full_name,
                        contact_number=contact_number,
                        email_address=email_address,
                        location=location,
                        technical_skills=technical_skills_str,
                        non_technical_skills=non_technical_skills_str,
                        education=education_str,
                        work_experience=work_experience_str,
                        certifications=certifications_str,
                        languages=languages_str,
                        suggested_resume_category=suggested_resume_category,
                        recommended_job_roles=recommended_job_roles_str)

# Start the Flask app

if __name__ == "__main__":
    app.run(debug=True)
```

BACKEND CODE

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Resume Uploader</title>

  <script src="https://cdn.tailwindcss.com"></script>

</head>

<body class="bg-gray-100 min-h-screen">

  <!-- Header -->

  <header class="bg-blue-600 text-white py-5 shadow-lg">

    <h1 class="text-center text-3xl font-bold">Resume Parsing with Google Gemini LLM</h1>

  </header>

  <!-- Container -->

  <div class="container mx-auto mt-10 p-6 bg-white shadow-lg rounded-lg">

    <h2 class="text-2xl font-semibold text-gray-700 mb-6">Upload Resume</h2>

    <!-- Upload Form -->

    <form id="resumeForm" action="/upload_resume" method="post"
enctype="multipart/form-data" class="space-y-4">

      <div class="flex flex-col">

        <label for="resume" class="text-gray-600 font-medium">Resume (PDF
only):</label>

        <input
```

```
        type="file"

        id="resume"

        name="resume"

        accept=".pdf"

        required

        class="file:mr-4 file:py-2 file:px-4 file:rounded-lg file:border-0 file:bg-blue-50
file:text-blue-600 hover:file:bg-blue-100 border border-gray-300 rounded-md py-2 px-4
focus:outline-none focus:ring-2 focus:ring-blue-400 focus:ring-opacity-50">

</div>

<button

    type="submit"

    class="w-full bg-blue-600 text-white py-2 rounded-lg hover:bg-blue-700 transition-
all duration-300 ease-in-out">

    Upload

</button>

</form>

{% if full_name %}

<!-- Extracted Resume Details -->

<div class="mt-10">

    <h2 class="text-xl font-semibold text-gray-700 mb-4">Extracted Resume
Details</h2>

    <form id="resumeDetailsForm" class="space-y-4">

        <!-- Full Name -->

        <div>

            <label for="fullName" class="text-gray-600 font-medium">Full Name:</label>
```

```
<input type="text" id="fullName" name="fullName" value="{{ full_name }}"
readonly class="w-full border border-gray-300 rounded-md py-2 px-4 bg-gray-100">
```

```
</div>
```

```
<!-- Contact Number -->
```

```
<div>
```

```
<label for="contactNumber" class="text-gray-600 font-medium">Contact
Number:</label>
```

```
<input type="text" id="contactNumber" name="contactNumber" value="{{
contact_number }}" readonly class="w-full border border-gray-300 rounded-md py-2 px-4
bg-gray-100">
```

```
</div>
```

```
<!-- Email Address -->
```

```
<div>
```

```
<label for="emailAddress" class="text-gray-600 font-medium">Email
Address:</label>
```

```
<input type="text" id="emailAddress" name="emailAddress" value="{{
email_address }}" readonly class="w-full border border-gray-300 rounded-md py-2 px-4 bg-
gray-100">
```

```
</div>
```

```
<!-- Location -->
```

```
<div>
```

```
<label for="location" class="text-gray-600 font-medium">Location:</label>
```

```
<input type="text" id="location" name="location" value="{{ location }}"
readonly class="w-full border border-gray-300 rounded-md py-2 px-4 bg-gray-100">
```

```
</div>
```

<!-- Technical Skills -->

<div>

<label for="technicalSkills" class="text-gray-600 font-medium">Technical Skills:</label>

<textarea id="technicalSkills" name="technicalSkills" rows="4" readonly class="w-full border border-gray-300 rounded-md py-2 px-4 bg-gray-100">{{ technical_skills }}</textarea>

</div>

<!-- Non-Technical Skills -->

<div>

<label for="nonTechnicalSkills" class="text-gray-600 font-medium">Non-Technical Skills:</label>

<textarea id="nonTechnicalSkills" name="nonTechnicalSkills" rows="4" readonly class="w-full border border-gray-300 rounded-md py-2 px-4 bg-gray-100">{{ non_technical_skills }}</textarea>

</div>

<!-- Education -->

<div>

<label for="education" class="text-gray-600 font-medium">Education:</label>

<textarea id="education" name="education" rows="4" readonly class="w-full border border-gray-300 rounded-md py-2 px-4 bg-gray-100">{{ education }}</textarea>

</div>

<!-- Work Experience -->

<div>

<label for="workExperience" class="text-gray-600 font-medium">Work Experience:</label>

```
<textarea id="workExperience" name="workExperience" rows="4" readonly
class="w-full border border-gray-300 rounded-md py-2 px-4 bg-gray-100">{{
work_experience }}</textarea>
```

```
</div>
```

```
<!-- Certifications -->
```

```
<div>
```

```
<label for="certifications" class="text-gray-600 font-
medium">Certifications:</label>
```

```
<textarea id="certifications" name="certifications" rows="4" readonly class="w-
full border border-gray-300 rounded-md py-2 px-4 bg-gray-100">{{ certifications
}}</textarea>
```

```
</div>
```

```
<!-- Languages -->
```

```
<div>
```

```
<label for="languages" class="text-gray-600 font-medium">Languages:</label>
```

```
<textarea id="languages" name="languages" rows="2" readonly class="w-full
border border-gray-300 rounded-md py-2 px-4 bg-gray-100">{{ languages }}</textarea>
```

```
</div>
```

```
<!-- Suggested Resume Category -->
```

```
<div>
```

```
<label for="suggestedCategory" class="text-gray-600 font-medium">Suggested
Resume Category:</label>
```

```
<input type="text" id="suggestedCategory" name="suggestedCategory"
value="{{ suggested_resume_category }}" readonly class="w-full border border-gray-300
rounded-md py-2 px-4 bg-gray-100">
```

```
</div>
```

```
<!-- Recommended Job Roles -->

<div>

  <label for="recommendedRoles" class="text-gray-600 font-
medium">Recommended Job Roles:</label>

  <textarea id="recommendedRoles" name="recommendedRoles" rows="2"
readonly class="w-full border border-gray-300 rounded-md py-2 px-4 bg-gray-100">{{
recommended_job_roles }}</textarea>

</div>

</form>

</div>

{% endif %}

</div>

<!-- Footer -->

<footer class="bg-gray-200 text-center py-4 mt-10">

  <p class="text-gray-600">&copy; 2024 Resume Parsing App. All rights reserved.</p>

</footer>

</body>

</html>
```

CHAPTER 4

RESULTS

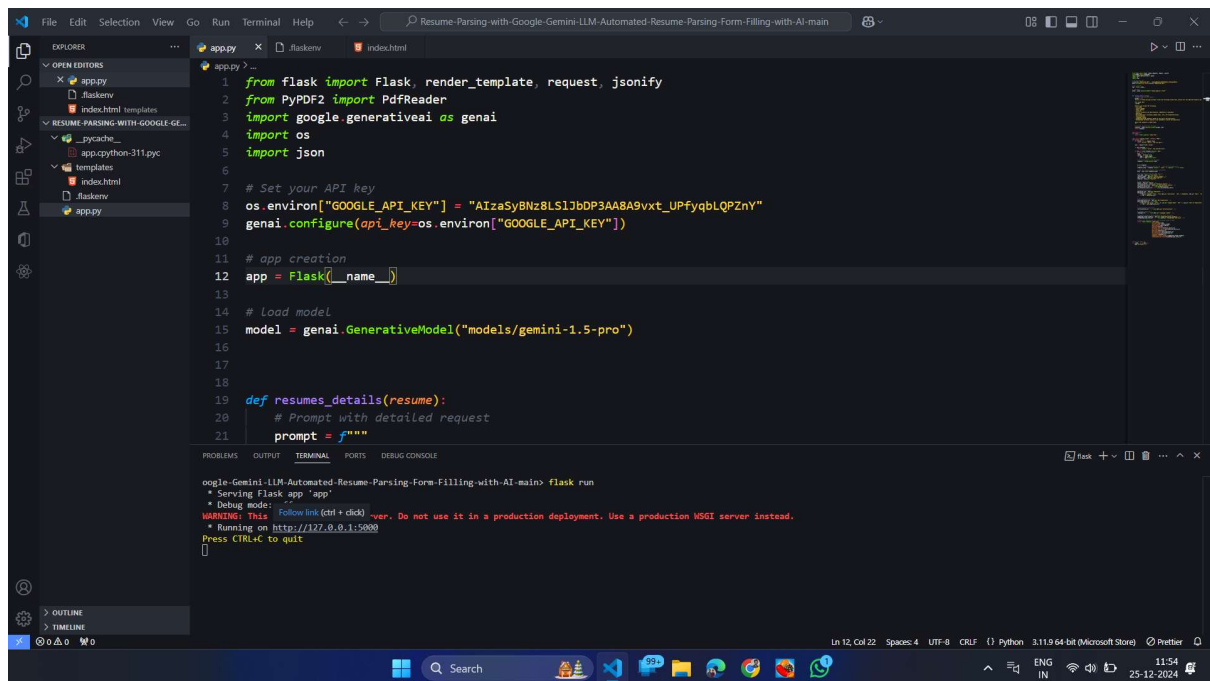
When you upload a resume, the app reads it and pulls out important details. It shows these details on the webpage so you can easily review them.

1. **Full Name:**
The name on the resume, like *John Doe*.
2. **Contact Number:**
Your phone number, such as *+91 234-567-8901*.
3. **Email Address:**
Your email, like johndoe@example.com.
4. **Location:**
The city or area you live in, such as *XYZ , INDIA*.
5. **Technical Skills:**
Job-related skills, like knowing programming languages or tools, for example, *Python, JavaScript, SQL*.
6. **Non-Technical Skills:**
Personal skills, like teamwork or communication, such as *Leadership, Communication*.
7. **Education:**
Your degrees and where you studied, like *B.Sc. in Computer Science from MIT (Graduated: 2021)*.
8. **Work Experience:**
Past jobs, including the company name, job title, and what you did there.
For example:
Software Engineer at Google (2018–2022), where you improved system performance.
9. **Certifications:**
Any special certificates you earned, like *AWS Certified Solutions Architect*.
10. **Languages:**
The languages you can speak, such as *English, Spanish*.
11. **Suggested Resume Category:**
What type of job your resume is best suited for, like *Data Scientist*.
12. **Recommended Job Roles:**
Ideas for specific jobs you might fit well into, such as *Machine Learning Engineer*.

Why This is Useful

- **Quick Review:** You can see everything important from your resume in one place.
- **Suggestions:** It tells you what jobs you might be a good match for.
- **Improvements:** If something is missing or unclear, you can fix it in your resume.

USER INTERFACE WITH RESULTS



```
1 from flask import Flask, render_template, request, jsonify
2 from PyPDF2 import PdfReader
3 import google.generativeai as genai
4 import os
5 import json
6
7 # Set your API key
8 os.environ["GOOGLE_API_KEY"] = "AIzaSy8Nz8LS1JbDP3AA8A9vxt_UFyqbLQPZny"
9 genai.configure(api_key=os.environ["GOOGLE_API_KEY"])
10
11 # app creation
12 app = Flask(__name__)
13
14 # Load model
15 model = genai.GenerativeModel("models/gemini-1.5-pro")
16
17
18
19 def resumes_details(resume):
20     # Prompt with detailed request
21     prompt = f"""
```

oogle-Gemini-LLM-Automated-Resume-Parsing-Form-Filling-with-AI-main flask run

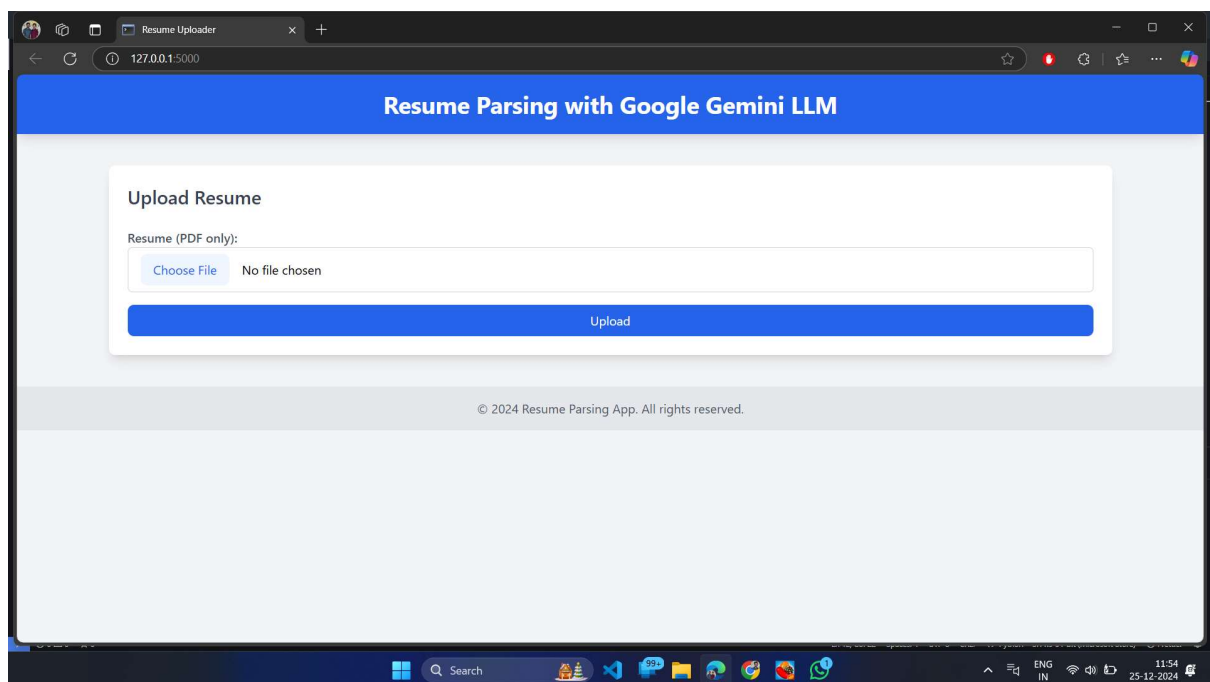
Serving Flask app 'app'

Debug mode: ON

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

Running on http://127.0.0.1:5000

Press CTRL+C to quit



Resume Parsing with Google Gemini LLM

Upload Resume

Resume (PDF only):

[Choose File](#) SuryaResume.pdf

[Upload](#)

© 2024 Resume Parsing App. All rights reserved.

Resume Parsing with Google Gemini LLM

Upload Resume

Resume (PDF only):

[Choose File](#) No file chosen

[Upload](#)

Extracted Resume Details

Full Name:
SURYAPRAKASHGOWDA H S

Contact Number:
+91 9901506415

Email Address:
suryagowda6415@gmail.com

Location:
Srinivasapur, Kolar Dist.

Resume Parsing with Google Gemini LLM

Education:

Artificial Intelligence and Machine Learning from Government Engineering College, Challakere (Graduated: 2022-2026)
PCMB from Gangotri PU College, Srinivasapur (Graduated: 2020-2022)
SSLC from SFS HIGH SCHOOL, Srinivasapur (Graduated: 2020)

Work Experience:

Certifications:

Languages:

Suggested Resume Category:

Entry-Level/Student in Artificial Intelligence and Machine Learning

Recommended Job Roles:

Junior Machine Learning Engineer, Data Scientist Intern, AI Research Assistant, Software Engineer Intern

Fig.no. 4.1

FUTURE WORK

1. Enhanced Resume Parsing & Extraction

- **Handle Multiple File Formats:** Currently, your system only supports PDF resumes. You could add support for other formats such as DOCX, TXT, and even images.

2. Advanced Data Visualization

- **More Graph Types:** Besides the pie chart, you could introduce other visualizations such as bar charts, skill trees, or even word clouds to highlight the most common keywords in the resume.
- **Work Experience Timeline:** Build a timeline visualizing the work experience and the career progression over time, which could help recruiters and hiring managers see the career growth clearly.

3. Backend Enhancements

- **Cloud Storage Integration:** Store uploaded resumes in a cloud service (like Google Cloud Storage, AWS S3, etc.) and allow users to view, download, or even share their processed resume with others.
- **Batch Processing:** Add the ability to process multiple resumes in a batch, which would be helpful for recruitment agencies.

4. User Interface Improvements

- **Resume Editing Interface:** After parsing, provide a web-based editor where users can manually correct any mistakes in the parsed data or fine-tune the layout.
- **Multi-Language Support:** Add the ability for the tool to process resumes in different languages, expanding your user base and making it more global.

5. AI-driven Resume Scoring & Benchmarking

- **Scoring System:** Introduce a scoring algorithm based on resume completeness, relevance of skills, and overall presentation. You could give the user a score or feedback that ranks their resume compared to others in the same field.
- **Benchmarking:** Compare the candidate's skills and qualifications against industry benchmarks to offer insights into how competitive their resume is.

6. Collaborative Features

- **Team Feedback:** Allow for a team of recruiters or colleagues to provide feedback on a particular resume. The system could then aggregate their feedback and suggest changes.
- **Job Portal Integration:** Integrate your system with job portals (LinkedIn, Indeed, etc.) so users can directly apply to jobs after parsing and improving their resumes.

7. Mobile App or Progressive Web App (PWA)

- **Mobile-Friendly Design:** Develop a mobile version or Progressive Web App (PWA) that allows users to upload, parse, and receive their resume insights directly from their phones.
- **Mobile Editing:** Allow users to edit or update their resumes on mobile devices, making it easier for job seekers on the go.

CHAPTER 5

CONCLUSION

“Resume Parsing with Google Gemini LLM “is automate the parsing of resumes, which greatly improves the recruitment process. By automatically extracting important candidate information, the system speeds up how quickly recruiters can gather and review applications. This means hiring decisions can be made more efficiently, allowing recruiters to spend more time on the actual evaluation of candidates.

The automation also helps reduce errors that can occur when entering candidate information manually. This leads to more accurate data, ensuring that qualifications are correctly represented. Additionally, the system works to eliminate biases in the evaluation process, making sure all candidates are assessed fairly based on their skills and experiences.

Candidates benefit from a smoother application experience, as the quicker recruitment process allows for faster feedback and decisions. This positive experience not only encourages candidates to apply but also enhances the organization’s image as an efficient and fair employer.

FUTURE SCOPE

The project could develop into a platform that helps users find job openings that match their resumes. It could also assist employers by making it easier for them to review candidates and understand what skills are in demand.

- Job Matching Platform
- AI-Driven Resume Recommendations
- Recruiter Tool
- Automated Job Application